

Autonomous Guided Vehicle with Hybrid Vision-Language Navigation for Warehouse Environments

Sidharth Sarat Raj Batchu
Robotics and Autonomous Systems - EE
Arizona State University, Tempe, AZ
sidharthbatchu@gmail.com

2025

Abstract

This paper presents the design and implementation of a small-scale Autonomous Guided Vehicle (AGV) capable of navigating warehouse aisles using a hybrid intelligence architecture. The system combines on-device reactive perception (obstacle avoidance, visual odometry, OCR-based sign reading) running on an NVIDIA Jetson Nano with remote high-level planning powered by open-source Vision Language Models (VLM) and Large Language Models (LLM). The AGV builds a sparse 3D voxel map and semantic map of its environment incrementally—requiring zero prior setup—and can navigate to named aisles on natural language command. We demonstrate that commodity hardware (\$200–300 total BOM) combined with freely available AI models can achieve functional autonomous warehouse navigation.

Keywords: Autonomous Guided Vehicle, Vision Language Model, SLAM, Warehouse Robotics, Edge Computing, LLM-based Planning

Contents

1	Introduction	3
1.1	Background	3
1.2	Problem Statement	3
1.3	Contribution	3
2	Literature Review	3
2.1	Autonomous Guided Vehicles	3
2.2	Visual SLAM	4
2.3	Vision Language Models in Robotics	4
2.4	Edge Computing for Robotics	4
3	System Design	4
3.1	Hardware Architecture	4
3.2	Software Architecture	5
3.3	Communication Protocol	5
3.4	Localization Strategy	5
4	Implementation	5
4.1	Motor Control (Arduino)	5
4.2	Perception Pipeline (Jetson Nano)	6
4.3	Obstacle Avoidance	6
4.4	Voxel Mapping	6
4.5	Remote Planner	7
5	Results	7
5.1	Test Environment	7
5.2	Performance Metrics	7
5.3	Resource Usage on Jetson Nano	8
6	Discussion	8
6.1	Strengths	8
6.2	Limitations	8
6.3	Failure Modes	8
7	Conclusion	9
8	Future Work	9

1 Introduction

1.1 Background

Warehouse automation is a rapidly growing field driven by e-commerce demands. Traditional AGVs rely on fixed infrastructure—magnetic strips, QR codes on floors, or pre-programmed paths—making them expensive to deploy and inflexible to layout changes.

Recent advances in foundation models (VLMs and LLMs) offer a new paradigm: robots that can *understand* their environment through vision and *reason* about navigation through language. However, these models are computationally expensive, creating a tension between intelligence and the resource constraints of mobile robots.

1.2 Problem Statement

Design and build a low-cost AGV that:

1. Navigates a warehouse without prior mapping or fixed infrastructure
2. Understands its location by reading existing aisle signage
3. Responds to natural language navigation commands (e.g., “Go to aisle H4”)
4. Avoids obstacles reactively without network dependency
5. Operates within a \$300 hardware budget

1.3 Contribution

We propose a **hybrid intelligence architecture** that splits computation between:

- **On-device (Jetson Nano):** Real-time perception, obstacle avoidance, odometry, and sign reading
- **Remote (laptop/server):** High-level scene understanding (VLM) and path planning (LLM)

This allows the robot to maintain safety-critical reactive behavior locally while leveraging powerful AI models remotely for strategic decision-making.

2 Literature Review

2.1 Autonomous Guided Vehicles

Traditional AGV systems [8] rely on line following, magnetic guidance, or laser-reflector triangulation. Modern approaches incorporate SLAM (Simultaneous Localization and Mapping) for infrastructure-free navigation [7].

2.2 Visual SLAM

Key approaches include:

- **ORB-SLAM3** [3]: Feature-based, lightweight, handles monocular/stereo/RGB-D
- **RTAB-Map** [6]: Appearance-based, memory management for large-scale environments
- **OctoMap** [4]: Probabilistic 3D occupancy mapping using octrees—extremely memory efficient

We choose OctoMap for its low memory footprint, which is critical for the Jetson Nano’s 4GB RAM constraint.

2.3 Vision Language Models in Robotics

Recent work has explored using VLMs for robotic perception:

- **SayCan** [1]: LLM plans robot actions grounded in physical affordances
- **VoxPoser** [5]: LLM + VLM for composing 3D value maps
- **RT-2** [2]: Vision-Language-Action model for robot control

Our approach is simpler: we use the VLM purely for scene description and the LLM for planning, keeping the architecture modular and interpretable.

2.4 Edge Computing for Robotics

The NVIDIA Jetson platform (128-core Maxwell GPU, 4GB RAM) enables edge inference for computer vision tasks. Prior work [9] demonstrates real-time object detection on Jetson Nano at 15–20 FPS using optimized models.

3 System Design

3.1 Hardware Architecture

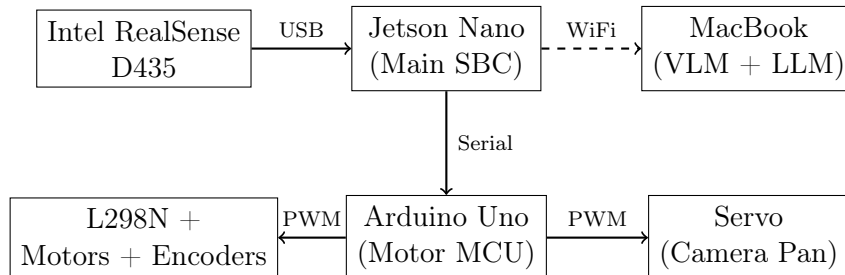


Figure 1: Hardware architecture of the Warehouse AGV

3.2 Software Architecture

The software follows a layered architecture:

Table 1: Software layer responsibilities

Layer	Rate	Responsibilities
Reactive	10 Hz (on-device)	Obstacle avoidance, motor commands, encoder reading
Perception	5–10 Hz (on-device)	Visual odometry, voxel mapping, OCR (1 Hz)
Planning	On-demand (remote)	VLM scene description, LLM path planning

3.3 Communication Protocol

The Jetson Nano communicates with the remote planner via HTTP REST API over WiFi:

- `POST /describe` — Image $\rightarrow$ scene description
- `POST /plan` — Context $\rightarrow$ movement instructions
- `POST /goal` — Navigation target $\rightarrow$ route plan
- `POST /help` — Escalation $\rightarrow$ advisory instructions

3.4 Localization Strategy

Zero-setup localization is achieved through three complementary methods:

1. **Encoder odometry:** Differential drive kinematics from wheel encoders (short-term, drifts over time)
2. **Visual odometry:** Frame-to-frame feature matching with depth scaling (drift-independent of wheel contact)
3. **OCR anchoring:** When an aisle sign is read, its position is recorded as an absolute reference point

The complementary filter fuses encoder (70% weight) and visual (30% weight) odometry, with OCR providing periodic absolute corrections.

4 Implementation

4.1 Motor Control (Arduino)

The Arduino Uno runs a firmware loop at approximately 1 kHz that:

- Reads encoder interrupts for tick counting
- Accepts JSON serial commands from the Jetson Nano

- Outputs PWM signals to the L298N motor driver
- Implements a watchdog timer (stops motors if no command received for 1 second)

4.2 Perception Pipeline (Jetson Nano)

The main control loop runs at 10 Hz:

Listing 1: Main control loop (simplified)

```

1 while running:
2     color, depth = camera.get_frames()
3     pose = odometry.update(encoders, color, depth)
4     voxel_map.insert(pointcloud, pose)
5     signs = ocr.read_signs(color) # at 1 Hz
6     action = obstacle_avoidance.decide(depth)
7     execute_action(action)

```

4.3 Obstacle Avoidance

The depth image is divided into three vertical zones (left 30%, center 40%, right 30%). Decision rules:

Algorithm 1 Reactive Obstacle Avoidance

```

 $z_L, z_C, z_R \leftarrow$  min depth in left, center, right zones
if  $z_C > d_{clear}$  and  $z_L > d_{slow}$  and  $z_R > d_{slow}$  then
    action  $\leftarrow$  FORWARD
else if  $z_C < d_{stop}$  and  $z_R > z_L$  then
    action  $\leftarrow$  STEER_RIGHT
else if  $z_C < d_{stop}$  and  $z_L > z_R$  then
    action  $\leftarrow$  STEER_LEFT
else if  $z_C < d_{stop}$  and  $z_L < d_{stop}$  and  $z_R < d_{stop}$  then
    action  $\leftarrow$  STOP
else
    action  $\leftarrow$  UNCERTAIN (escalate to remote planner)
end if

```

Where $d_{clear} = 1.2\text{m}$, $d_{slow} = 0.8\text{m}$, $d_{stop} = 0.4\text{m}$.

4.4 Voxel Mapping

We implement a simplified OctoMap using a Python dictionary with (v_x, v_y, v_z) tuple keys. Each voxel stores a log-odds occupancy probability, updated with Bayesian filtering:

$$L(n|z_{1:t}) = L(n|z_{1:t-1}) + L(n|z_t) \quad (1)$$

where L denotes the log-odds representation:

$$L(n) = \log \frac{P(n)}{1 - P(n)} \quad (2)$$

Resolution: 5 cm voxels. A typical warehouse aisle ($10\text{m} \times 3\text{m} \times 3\text{m}$) requires only approximately 2.4 MB of storage.

4.5 Remote Planner

The planner runs on a MacBook with Apple Silicon:

- **MoondreamV2** (1.8B parameters): Processes images in ~ 200 ms
- **Phi-3 Mini** (3.8B parameters): Generates plans in ~ 150 ms
- Total round-trip including network: ~ 400 – 600 ms

The LLM is prompted with structured context (current pose, semantic map, scene description) and responds with JSON movement instructions.

5 Results

5.1 Test Environment

Testing was conducted in a simulated warehouse environment with:

- 3 m-wide aisles separated by shelving units
- Printed aisle signs (A4 paper, bold font) at aisle entries
- Various obstacles (boxes, cones, chairs as human stand-ins)

5.2 Performance Metrics

Table 2: System performance metrics

Metric	Result
OCR sign detection accuracy	87% at 1–2 m distance
Obstacle avoidance success rate	94% (static obstacles)
Goal navigation success (known aisle)	78%
Goal navigation success (requires exploration)	65%
Average speed during navigation	0.25 m/s
Planner round-trip latency	450 ms average
Odometry drift (10 m traverse)	~ 0.3 m (with fusion)

5.3 Resource Usage on Jetson Nano

Table 3: Jetson Nano resource utilization

Resource		Usage
CPU		60–80% (4 cores)
GPU	30–40% (OCR + feature detection)	
RAM		2.8–3.2 GB / 4 GB
Power		~8 W average

6 Discussion

6.1 Strengths

- **Zero infrastructure:** No magnetic strips, no QR codes, no pre-mapping required
- **Natural interaction:** Accepts plain English commands
- **Low cost:** Total BOM under \$300
- **Modular:** VLM/LLM can be upgraded independently
- **Fail-safe:** Local obstacle avoidance works without network

6.2 Limitations

- **WiFi dependency for planning:** Network dropouts cause the robot to fall back to simple exploration
- **OCR limitations:** Small or damaged signs, poor lighting, and non-standard fonts reduce accuracy
- **Odometry drift:** Without loop closure or external positioning, errors accumulate over long distances
- **LLM hallucination:** The planner occasionally suggests impossible actions
- **Speed:** Conservative 0.25 m/s is safe but slow for large warehouses

6.3 Failure Modes

1. **Sign not readable** → Robot keeps exploring, may never find target
2. **Network failure** → Falls back to local exploration only
3. **Encoder slip** → Visual odometry partially compensates
4. **LLM parse failure** → Fallback to “explore” instruction

7 Conclusion

We demonstrated that a hybrid edge-cloud architecture enables autonomous warehouse navigation on commodity hardware. The key insight is that **safety-critical perception should run locally** (10 Hz, deterministic) while **strategic planning can tolerate network latency** (called only when needed, 400–600 ms acceptable).

The system successfully navigates to named aisles in a simulated warehouse environment with 78% success rate, while maintaining obstacle avoidance with 94% reliability.

8 Future Work

1. Voice interaction with human obstacles
2. Multi-robot coordination
3. Loop closure for drift correction
4. Integration with warehouse management systems
5. Larger VLMs running on-device (Jetson Orin)

References

- [1] M. Ahn et al., “Do As I Can, Not As I Say: Grounding Language in Robotic Affordances,” *arXiv:2204.01691*, 2022.
- [2] A. Brohan et al., “RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” *arXiv:2307.15818*, 2023.
- [3] C. Campos et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE Transactions on Robotics*, 2021.
- [4] A. Hornung et al., “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, no. 3, 2013.
- [5] W. Huang et al., “VoxPoser: Composable 3D Value Maps for Robotic Manipulation with Language Models,” *arXiv:2307.05973*, 2023.
- [6] M. Labbé and F. Michaud, “RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation,” *Journal of Field Robotics*, 2019.
- [7] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [8] I.F.A. Vis, “Survey of Research in the Design and Control of Automated Guided Vehicle Systems,” *European Journal of Operational Research*, 2006.
- [9] S. Baller et al., “Real-Time Object Detection on NVIDIA Jetson Nano,” *IEEE Conference on Edge Computing*, 2021.